

(Empty page. Do not put any answers here.)

This exam consists of 17 questions.

Points are indicated for each question and broken down to each part.

Question 1: Software Engineering (3 points)

List two aspects of how software engineering is different from other engineering disciplines. Illustrate each difference on an example.

Question 2: Simple Programs vs. Software Products (6 points)

List and briefly explain three aspects from the lecture how simple *programs* are different from *software products*. For each aspect explain if and how SCRUM reflects the difference.

Question 3: Rational Unified Process vs. Waterfall Model (6 points)

Pick three aspects of the Rational Unified Process (RUP), contrast them with the Waterfall model, and explain the intended benefit of each aspect of the RUP.

Question 4: Software Engineering Mistakes (6 points)

List 3 classic software engineering mistakes from the lecture and explain for each how the Rational Unified Process addresses it.

Question 5: Stakeholders (4 points)

Name four stakeholders (at least two external stakeholders) for a software for a car repair shop and provide for each a relevant concern for the software.

Question 6: Requirements classification (4 points)

For each of the following requirements, mark if they are functional (F) or non-functional (N):

- (a) ____ The system should support concurrent access by at least 1000 users.
- (b) ____ The system should process online payments.
- (c) ____ The system should encrypt sensitive customer data, such as credit card information.
- (d) ____ The system's database should be backed up every night at 2:00 AM.

Question 7: User stories (4 points)

Write two user stories as requirements for a car repair shop system.

Question 8: Use Case Diagram vs. Class Diagram (6 points)

List two commonalities and two differences between Use Case Diagrams and Class Diagrams. Give an example where one diagram type would be used over the other.

Question 9: Incompleteness (4 points)

Define incompleteness in requirements engineering. Give an example requirement where incompleteness cannot be avoided and explain why.

Question 10: Modeling Language Elements (6 points)

List and explain (max. one sentence) the three elements of a modeling language definition. Illustrate them on feature models.

Question 11: Implementation guidelines (6 points)

List three guidelines to follow during the implementation of a system and explain for each how it supports the development process.

Question 12: Testing techniques (4 points)

Describe one scenario where black-box testing and one scenario where white-box testing should be applied. Justify each answer.

Question 13: Combinatorial test design (8 points)

Use the In-Parameter-Order algorithm to complete the test plan below for pairwise coverage ($t = 2$). The parameters are $A = \{a_1, a_2\}$, $B = \{b_1, b_2\}$, and $C = \{c_1, c_2, c_3\}$.

 $a_1 \quad b_1$ $a_1 \quad b_2$ $a_2 \quad b_1$ $a_2 \quad b_2$ **Question 14: Metamorphic testing** (4 points)

Explain how an oracle can be (partially) constructed using metamorphic testing on the example of an exam grading system. Clearly state the metamorphic relation you assume to hold.

Question 15: White-box Testing

- (a) (6 points) Draw the control flow graph for the given code snippet (the lines are spaced to draw the graph next to the code). Labeling nodes with the line number is sufficient.

```
1 public static boolean check(int x) {  
2     if (x <= 3) {  
3         return false;  
4     }  
5     boolean y = true;  
6     for (int i = 0; i < x && y; i++) {  
7         if (x % i == 0) {  
8             y = false;  
9         }  
10    }  
11    return y;  
12 }
```

- (b) (2 points) Provide a minimal set of test cases for 100% statement coverage.

- (c) (3 points) Provide a minimal set of test cases for 100% branch coverage.

- (d) (2 points) Explain the relation between branch coverage and MC/DC coverage.

(e) (2 points) Compute the cyclomatic complexity of the program.

Question 16: Extreme Programming (4 points)

Name and briefly describe two practices of Extreme Programming.

Question 17: Code Quality (4 points)

Define the Law of Demeter. Illustrate a violation and state why violations may be a problematic.

(This is an additional sheet for your answers.)

(This is an additional sheet for your answers.)